Numerical Methods & Applied Programming (ENG2124)

Matplotlib for Visualization

Dr Ian (Yinglong) He

A: 14b AC 03

E: yh0015@surrey.ac.uk

T: 01483 688784

Content

- What is Matplotlib?
- Simple line/scatter plot
- Multiple subplots
- Bar plot
- Pie plot
- Histogram
- Box plot
- Image plot
- 3D Plot

What is Matplotlib?

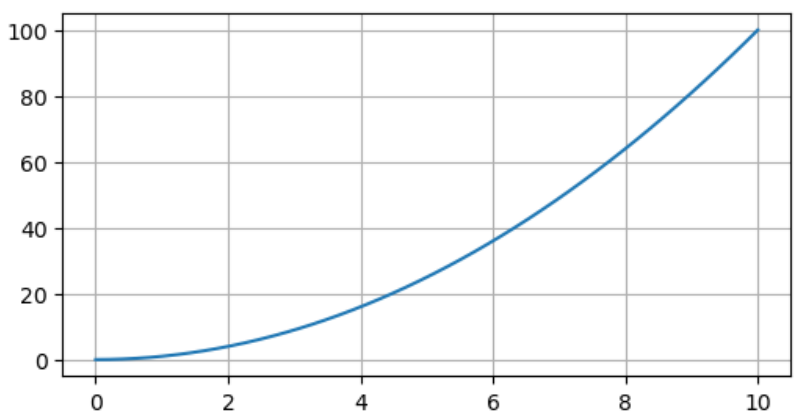
- Matplotlib is a graph plotting package, for creating static, animated, and interactive visualizations in Python.
- Matplotlib was created by John D. Hunter in 2003.
- Source code: https://github.com/matplotlib/matplotlib.
- It works well with NumPy.
- Its syntax is similar to MATLAB.

What is Matplotlib?

- Methods:
  - A simple line graph can be plotted with `plot()`.
  - A scatter plot can be created with `scatter()`.
  - A histogram can be created with `hist()`.
  - A bar chart can be created with `bar()`.
  - A pie chart can be created with `pie()`.
  - Plus many more …

Simple line/scatter plot

- Set figure size
- Create a line plot for $y = x^2$
- Show grid lines
- Save the figure

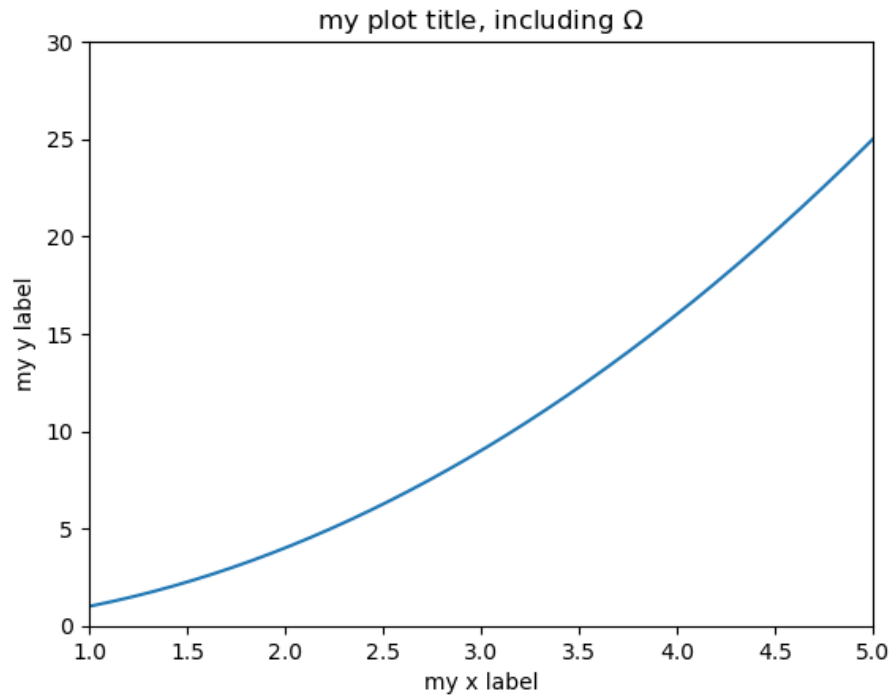



```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y = np.power(x, 2)

fig = plt.figure(figsize=(6, 3))

plt.plot(x, y)

plt.grid(True)

plt.savefig('my_figure.jpg')

plt.show()
```

Simple line/scatter plot

- Adjust axes limits
- Add axes labels and plot title

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y = np.power(x, 2)

plt.plot(x, y)

plt.xlim((1, 5))
plt.ylim((0, 30))

plt.xlabel('my x label')
plt.ylabel('my y label')

plt.title('my plot title, including $\Omega$')

plt.show()
```

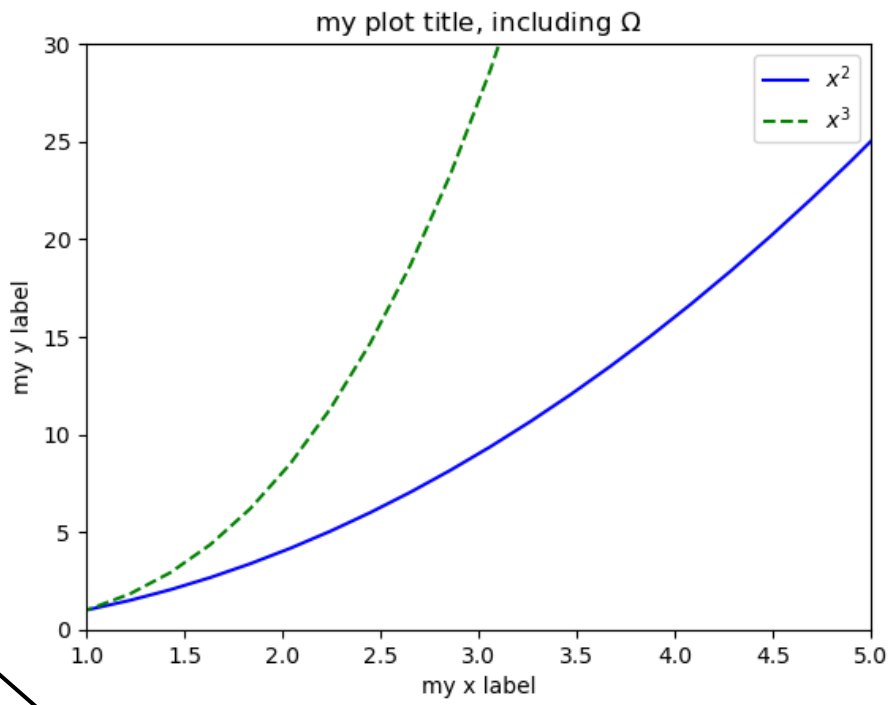


Further reading: *Writing mathematical expressions (LaTex, https://matplotlib.org/stable/tutorials/text/mathtext.html)*

Simple line/scatter plot

- Add multiple lines and adjust line colours and styles
- Show and position the legends

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y1 = np.power(x, 2)
y2 = np.power(x, 3)

plt.plot(x, y1, 'b-' , label='$x^2$')
plt.plot(x, y2, 'g--' , label='$x^3$')

plt.legend()

plt.xlim((1, 5))
plt.ylim((0, 30))

plt.xlabel('my x label')
plt.ylabel('my y label')

plt.title('my plot title, including $\Omega$')

plt.show()
```

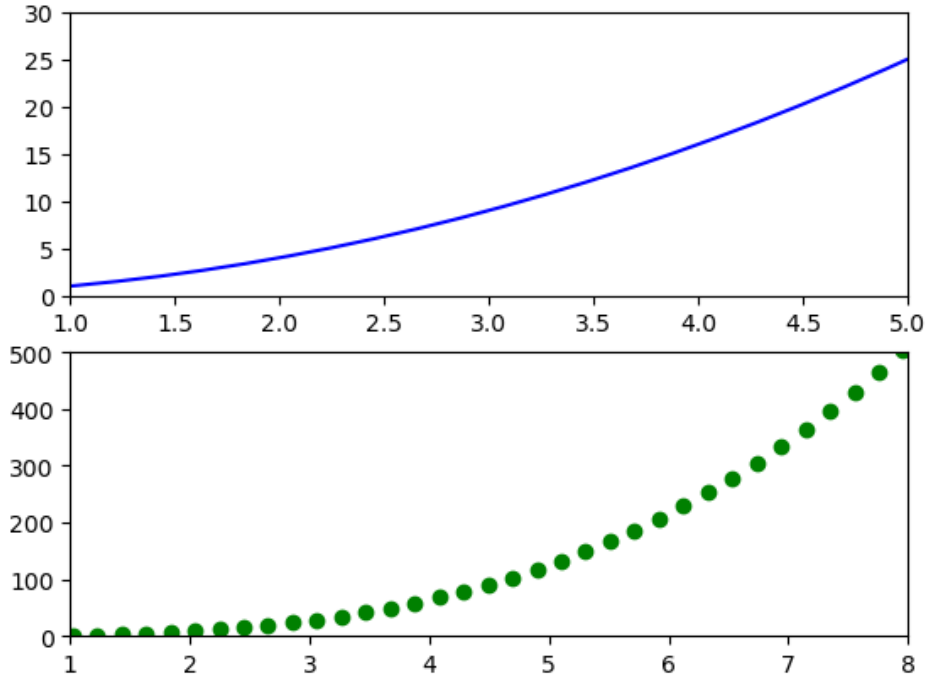


Further reading: *line styles, markers, colours* (https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html)

Multiple subplots

- Use subplot() and *MATLAB-style* (plt) functions to display multiple plots together.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y1 = np.power(x, 2)
y2 = np.power(x, 3)

plt.subplot(211)
plt.plot(x, y1)
plt.xlim((1, 5))
plt.ylim((0, 30))

plt.subplot(212)
plt.scatter(x, y2)
plt.xlim((1, 8))
plt.ylim((0, 500))

plt.show()
```

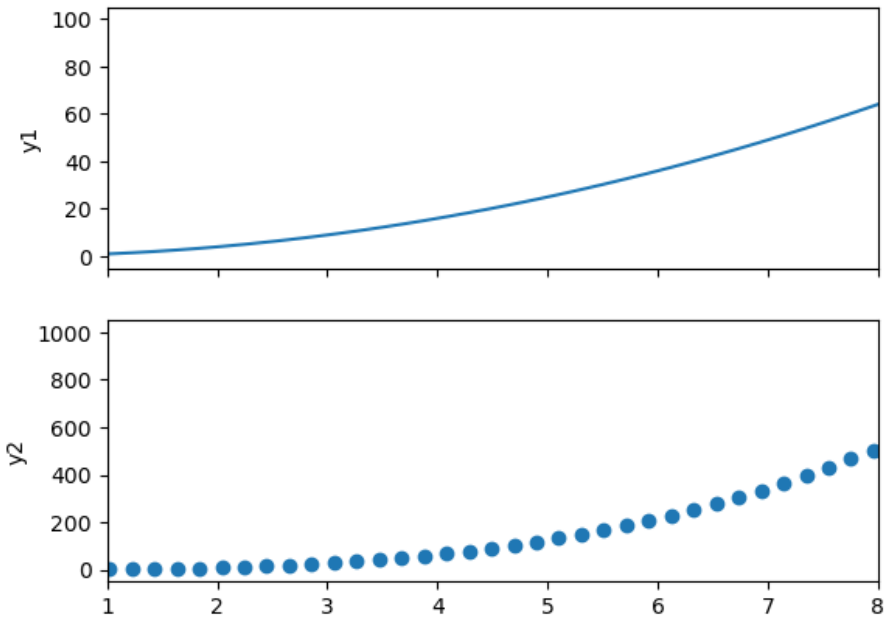


subplot(211): Create a 2 x 1 (row x col) grid for subplots and then draw on the 1st subplot.

Multiple subplots

- Use subplot() and *object-oriented* (ax) *methods*
- Share x axis

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y1 = np.power(x, 2)
y2 = np.power(x, 3)

fig, ax = plt.subplots(2, 1, sharex=True)

ax[0].plot(x, y1)
ax[0].set_ylabel('y1')

ax[1].scatter(x, y2)
ax[1].set_ylabel('y2')
ax[1].set_xlim((1, 8))

plt.show()
```

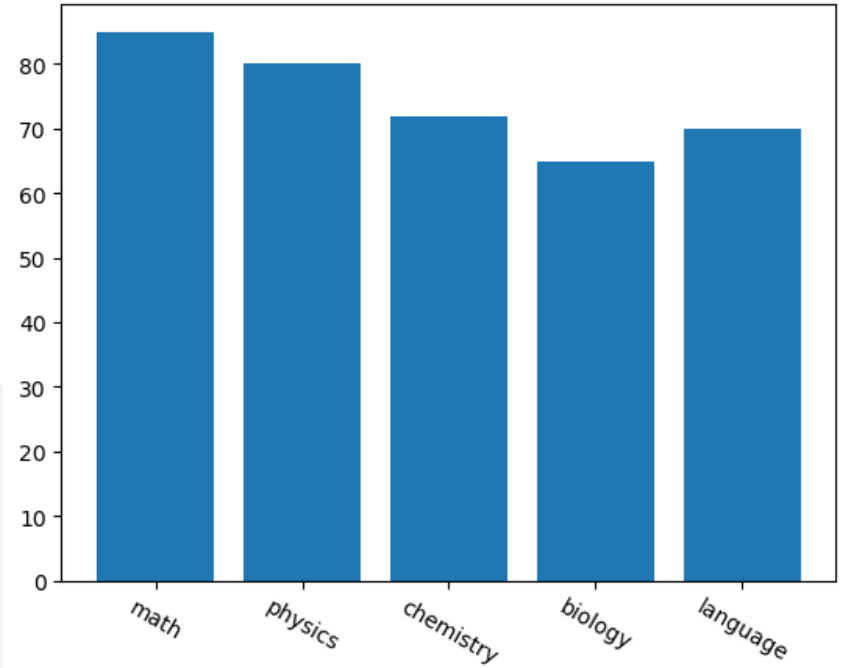


- Aside: Matplotlib MATLAB-style (plt) functions vs object-oriented (ax) methods

```
plt.xlabel()    →    ax.set_xlabel()
plt.ylabel()    →    ax.set_ylabel()
plt.xlim()      →    ax.set_xlim()
plt.ylim()      →    ax.set_ylim()
plt.title()     →    ax.set_title()
…
```

Bar plot

- Make a bar plot.
- Give labels at the given ticks locations.



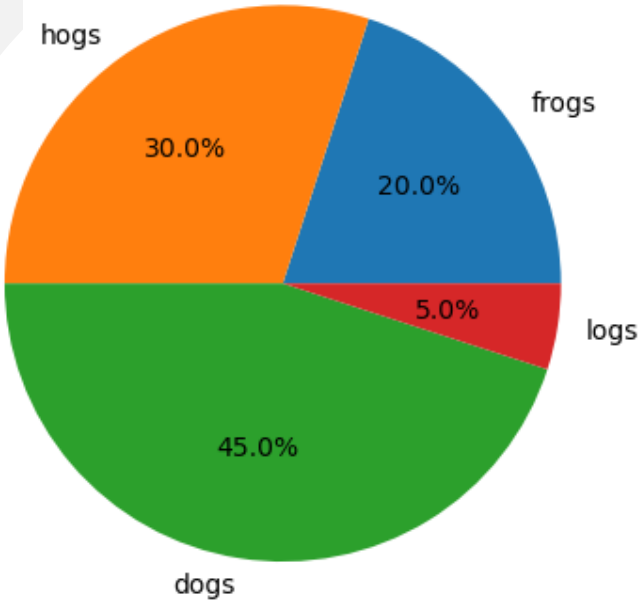

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [85, 80, 72, 65, 70]

plt.bar(x, y, align='center')

plt.xticks(x, ['math', 'physics', 'chemistry',
               'biology', 'language'], rotation=-30)

plt.show()
```

Pie plot

- Make a pie chart of array x. The fractional area of each wedge is given by x/sum(x).

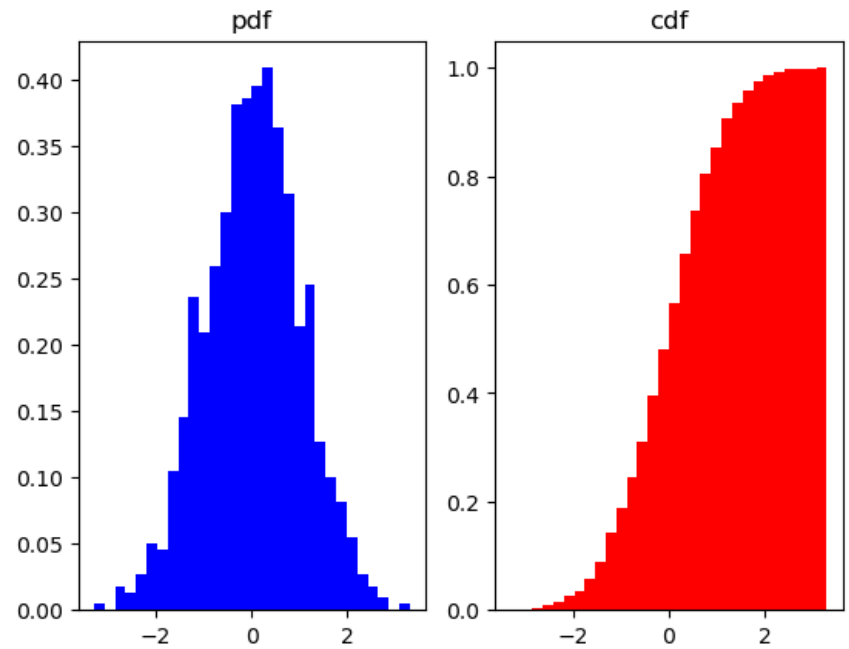
```
import matplotlib.pyplot as plt

labels = 'frogs', 'hogs', 'dogs', 'logs'
sizes = [40, 60, 90, 10]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')

plt.show()
```

Determine the decimal place of the data label in the pie chart.



Histogram

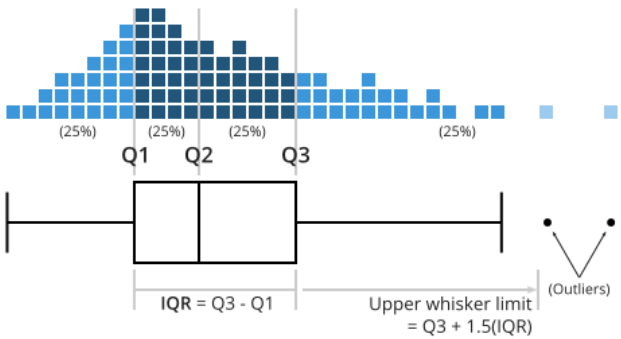
- Compute and plot a histogram, an approximate representation of the distribution of data.

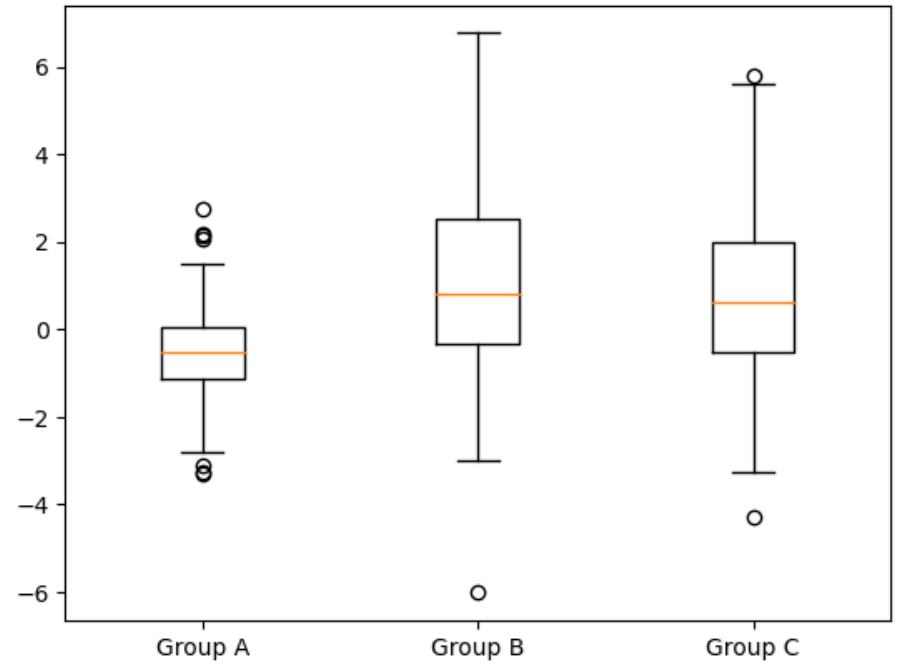



```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.randn(1000)

# histogram probability density function (pdf)
plt.subplot(121)
plt.hist(data , bins=30, density=True, color='b')
plt.title('pdf')

# empirical cumulative distribution function (cdf)
plt.subplot(122)
plt.hist(data , bins=30, density=True, color='r', cumulative=True)
plt.title('cdf')

plt.show()
```

Box plot

- Make a box and whisker plot.



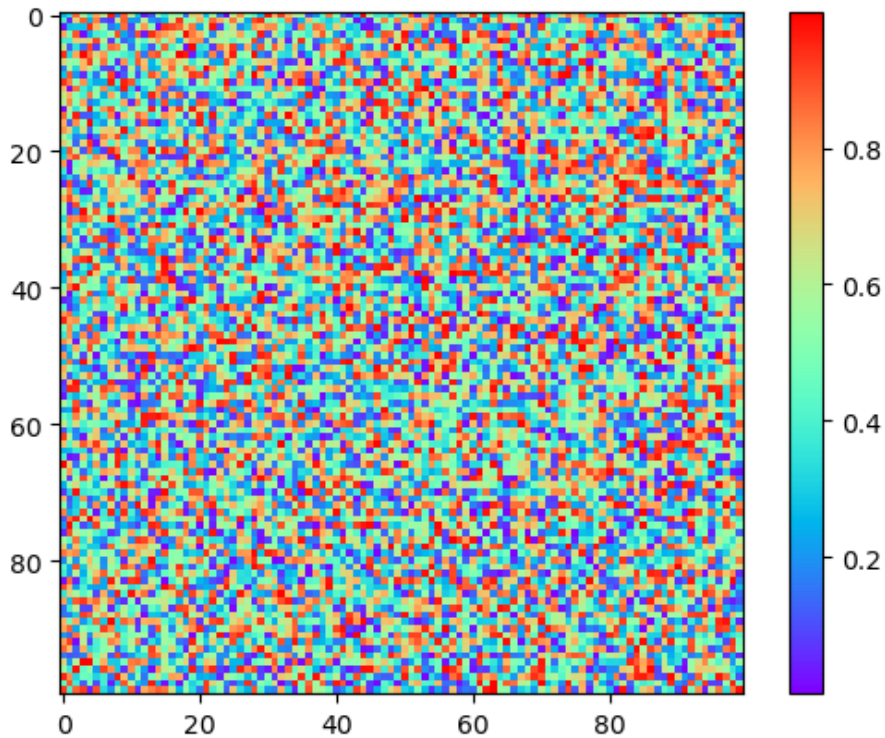



```
import numpy as np
import matplotlib.pyplot as plt

A = np.random.normal(loc=-0.5, scale=1., size=200)
B = np.random.normal(loc=1., scale=2., size=200)
C = np.random.normal(loc=0.5, scale=1.8, size=200)

plt.boxplot((A, B, C))

plt.xticks([1, 2, 3], ['Group A', 'Group B', 'Group C'])

plt.show()
```

Image plot

- Plot image data (2D array) and chose colormap (`cmap`)
- Show colour bar

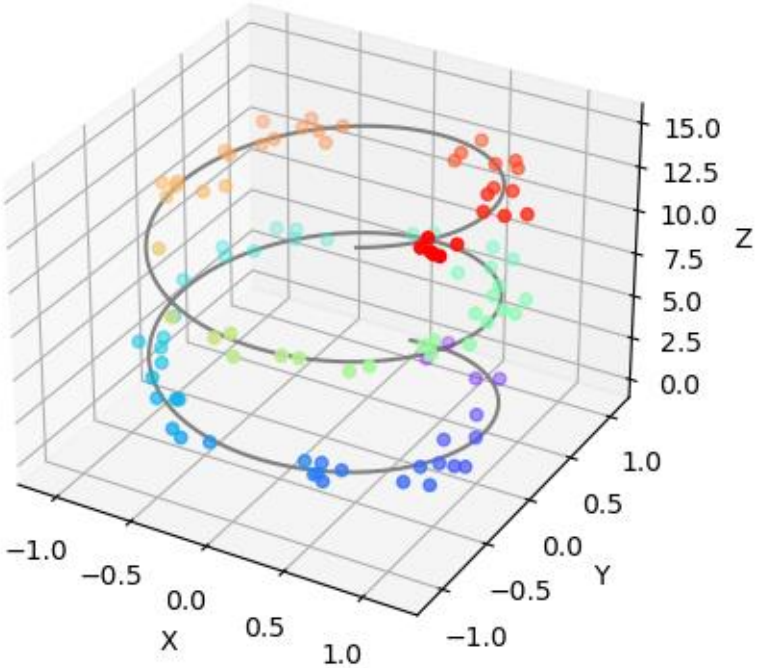
```
import numpy as np
import matplotlib.pyplot as plt

a = np.random.random((100, 100))

plt.imshow(a, cmap='rainbow')

plt.colorbar()

plt.show()
```



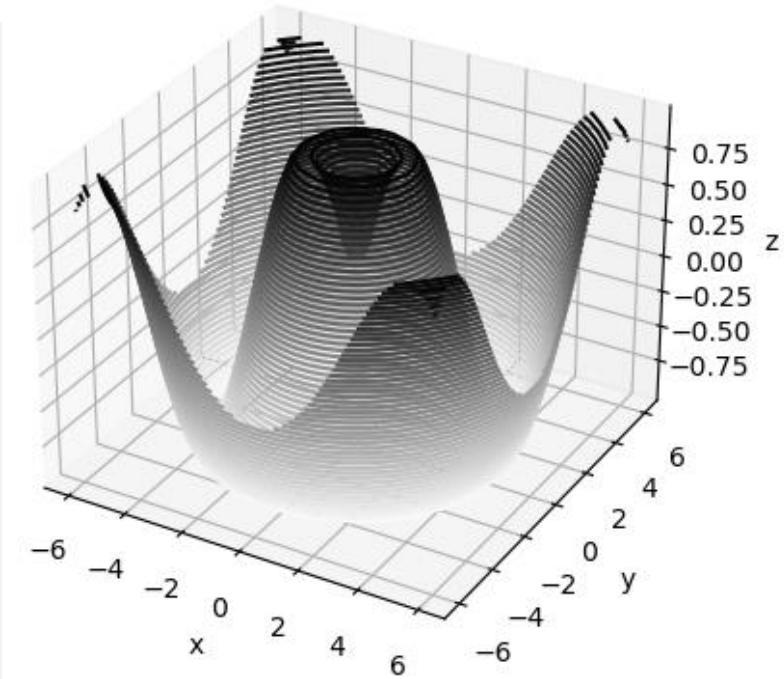
3D Plot

- 3D lines and points

```
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()
ax = plt.axes(projection='3d')

# data for a 3D line
zline = np.linspace(0, 15, 1000)
xline = np.sin(zline)
yline = np.cos(zline)

ax.plot3D(xline, yline, zline, 'gray')

# data for 3D scattered points
zdata = 15 * np.random.random(100)
xdata = np.sin(zdata) + 0.1 * np.random.randn(100)
ydata = np.cos(zdata) + 0.1 * np.random.randn(100)

ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='rainbow')

ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z')

plt.show()
```



3D Plot

- 3D contour plot for $z = \sin(\sqrt{x^2 + y^2})$

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y):
    return np.sin(np.sqrt(x ** 2 + y ** 2))

x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)

fig = plt.figure()
ax = plt.axes(projection='3d')

ax.contour3D(X, Y, Z, 50, cmap='binary')

ax.set_xlabel('x'); ax.set_ylabel('y'); ax.set_zlabel('z')

plt.show()
```